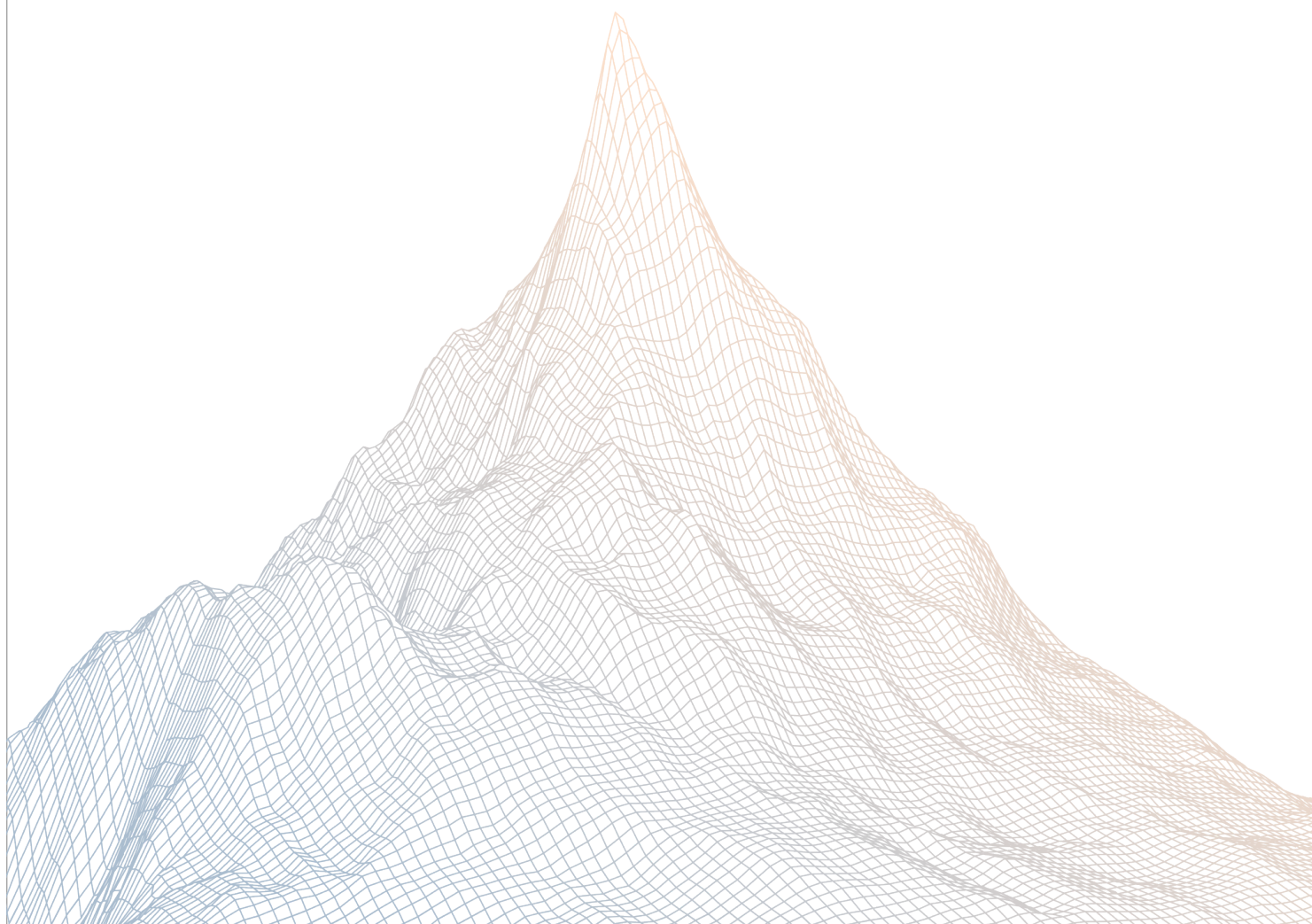


# Cosmos Labs

## Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: February 20th to March 17th, 2026  
AUDITED BY: Bernd  
Christian Vari

## Contents

|          |                          |          |
|----------|--------------------------|----------|
| <b>1</b> | <b>Introduction</b>      | <b>2</b> |
| 1.1      | About Zenith             | 3        |
| 1.2      | Disclaimer               | 3        |
| 1.3      | Risk Classification      | 3        |
| <hr/>    |                          |          |
| <b>2</b> | <b>Executive Summary</b> | <b>3</b> |
| 2.1      | About Cosmos Labs        | 4        |
| 2.2      | Scope                    | 4        |
| 2.3      | Audit Timeline           | 6        |
| 2.4      | Issues Found             | 6        |
| <hr/>    |                          |          |
| <b>3</b> | <b>Findings Summary</b>  | <b>6</b> |
| <hr/>    |                          |          |
| <b>4</b> | <b>Findings</b>          | <b>9</b> |
| 4.1      | Critical Risk            | 10       |
| 4.2      | High Risk                | 12       |
| 4.3      | Medium Risk              | 16       |
| 4.4      | Low Risk                 | 27       |
| 4.5      | Informational            | 50       |

# 1

## Introduction

### 1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

### 1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

### 1.3 Risk Classification

| SEVERITY LEVEL     | IMPACT: HIGH | IMPACT: MEDIUM | IMPACT: LOW |
|--------------------|--------------|----------------|-------------|
| Likelihood: High   | Critical     | High           | Medium      |
| Likelihood: Medium | High         | Medium         | Low         |
| Likelihood: Low    | Medium       | Low            | Low         |

## 2

### Executive Summary

## 2.1 About Cosmos Labs

Cosmos powers the largest network of vertically integrated Layer 1 blockchains in the world: 200+ blockchains use the Cosmos Stack in production. Cosmos Labs is responsible for developing new software and products for the Cosmos Stack and its customers, building and maintaining customer relationships, growing the adoption of our stack, and fostering the community of independent builders and end users using our open-source products.

## 2.2 Scope

The engagement involved a review of the following targets:

|                    |                                                                                                                                                                                                                                                                                                     |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Target</b>      | solidity-ibc-eureka                                                                                                                                                                                                                                                                                 |
| <b>Repository</b>  | <a href="https://github.com/cosmos/solidity-ibc-eureka.git">https://github.com/cosmos/solidity-ibc-eureka.git</a>                                                                                                                                                                                   |
| <b>Commit Hash</b> | 5560586d241a7e33f2359bdd0bed4403cee34ce5                                                                                                                                                                                                                                                            |
| <b>Files</b>       | programs/solana/programs/access-manager/**/*<br>programs/solana/programs/ics26-router/**/*<br>programs/solana/programs/ics27-gmp/**/*<br>programs/solana/programs/ift/**/*<br>programs/solana/packages/**/*<br>programs/solana/programs/attestation/**/*<br>packages/tendermint-light-client/solana |

|                    |                                                                                                                                                                                                                                                                                              |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Target</b>      | Mitigation Review                                                                                                                                                                                                                                                                            |
| <b>Repository</b>  | <a href="https://github.com/cosmos/solidity-ibc-eureka.git">https://github.com/cosmos/solidity-ibc-eureka.git</a>                                                                                                                                                                            |
| <b>Commit Hash</b> | 3b594aab7d17dcf5f1afe088a5ae8d95de695d7d                                                                                                                                                                                                                                                     |
| <b>Files</b>       | <pre>programs/solana/programs/access-manager/**/* programs/solana/programs/ics26-router/**/* programs/solana/programs/ics27-gmp/**/* programs/solana/programs/ift/**/* programs/solana/packages/**/* programs/solana/programs/attestation/**/* packages/tendermint-light-client/solana</pre> |

## 2.3 Audit Timeline

|                          |                  |
|--------------------------|------------------|
| <b>February 20, 2026</b> | Audit start      |
| <b>March 17, 2026</b>    | Audit end        |
| <b>April 8, 2026</b>     | Report published |

## 2.4 Issues Found

| SEVERITY            | COUNT     |
|---------------------|-----------|
| Critical Risk       | 1         |
| High Risk           | 2         |
| Medium Risk         | 7         |
| Low Risk            | 17        |
| Informational       | 10        |
| <b>Total Issues</b> | <b>37</b> |

## 3

## Findings Summary

| ID  | Description                                                                                                | Status       |
|-----|------------------------------------------------------------------------------------------------------------|--------------|
| C-1 | Attacker-controlled payer_position injects relayer's payer signer into arbitrary CPI, enabling fund theft  | Resolved     |
| H-1 | Ed25519 pre[]verification PDAs trusted as final truth in Solana Tendermint light client                    | Resolved     |
| H-2 | Relayer-controlled metadata port routes packets to the wrong IBC application, breaking core routing safety | Resolved     |
| M-1 | Shared ClientSequence counter causes stale PDA derivation and packet send DoS                              | Resolved     |
| M-2 | Pre-funded packet commitment PDA blocks send_packet for targeted senders                                   | Resolved     |
| M-3 | IFT refund mints are subject to rate limit and pause state, contrary to spec                               | Acknowledged |
| M-4 | Hardcoded GMP encoding constant prevents Solana-to-EVM IFT transfers                                       | Resolved     |
| M-5 | PendingTransfer PDA creation can be grieved via lamport prefunding                                         | Resolved     |
| M-6 | Anchor's Borsh-wrapped return data corrupts acknowledgement bytes delivered to source chain IBC apps       | Resolved     |
| M-7 | No pause/unpause mechanism for router despite documented emergency roles                                   | Resolved     |
| L-1 | Insufficient port ID validation in add_ibc_app                                                             | Resolved     |
| L-2 | Initialization of Solana programs can be frontrun to set an arbitrary admin                                | Resolved     |
| L-3 | Reserved role IDs are documented as unimplemented but remain fully assignable                              | Resolved     |
| L-4 | Missing grantee address validation allows assigning roles to zero address                                  | Resolved     |
| L-5 | Missing admin sanity check can permanently brick governance                                                | Resolved     |
| L-6 | Removing/deactivating bridge can permanently block pending-transfer finalization                           | Resolved     |

| ID   | Description                                                                                                            | Status       |
|------|------------------------------------------------------------------------------------------------------------------------|--------------|
| L-7  | Admin can brick minting but app state stays “live”                                                                     | Resolved     |
| L-8  | No explicit max-account bound in GmpSolanaPayload                                                                      | Resolved     |
| L-9  | Missing validation of access_manager pubkey during initialization                                                      | Resolved     |
| L-10 | Single-step admin transfer                                                                                             | Resolved     |
| L-11 | MIN_TIMEOUT_DURATION constant defined but never enforced in IFT transfer validation                                    | Resolved     |
| L-12 | Privileged ID_CUSTOMIZER_ROLE role can permanently block client registration through PDA seed collision                | Resolved     |
| L-13 | IFT mint creation can be grieved via lamport pre-funding of the mint account                                           | Resolved     |
| L-14 | Lack of attester set rotation mechanism                                                                                | Acknowledged |
| L-15 | Missing router-side owner check for consensus_state in send_packet                                                     | Resolved     |
| L-16 | verify_membership does not reject zero-commitment attestations, blurring the boundary with non-membership proofs       | Resolved     |
| L-17 | Attestation light client status never expires, lacking a liveness check on stale consensus states                      | Acknowledged |
| I-1  | Program whitelist accepts duplicate entries, reducing effective capacity                                               | Resolved     |
| I-2  | Unused ProgramExtendedEvent event                                                                                      | Resolved     |
| I-3  | Documentation inconsistency regarding CPI restrictions in upgrade_program instruction                                  | Resolved     |
| I-4  | Reduce MAX_CLIENT_ID_LENGTH to avoid exceeding Solana’s PDA seed limit                                                 | Resolved     |
| I-5  | Packet reconstruction validate_and_reconstruct_packet function accepts empty payloads due to NAND check instead of XOR | Resolved     |

---

| ID   | Description                                                                                                           | Status   |
|------|-----------------------------------------------------------------------------------------------------------------------|----------|
| I-6  | IFT_PORT_ID and IFT_VERSION constants are defined but never used                                                      | Resolved |
| I-7  | IBC app callback failures cannot be converted into universal error acknowledgements                                   | Resolved |
| I-8  | Packet.timeout_timestamp uses i64 on Solana while Solidity uses uint64, leading to an unsafe cast in the timeout path | Resolved |
| I-9  | Missing account reload after error-ack refund in IFT finalize_transfer                                                | Resolved |
| I-10 | IBCApp authority field is stored but never used for access control                                                    | Resolved |

---

# 4

## Findings

### 4.1 Critical Risk

A total of 1 critical risk findings were identified.

[C-1] Attacker-controlled payer\_position injects relayer's payer signer into arbitrary CPI, enabling fund theft

SEVERITY: Critical

IMPACT: High

STATUS: Resolved

LIKELIHOOD: High

#### Target

- [programs/solana/programs/ics27-gmp/src/instructions/on\\_recv\\_packet.rs#L178-L186](#)

#### Description:

The on\_recv\_packet instruction handler in the ics27-gmp program builds a CPI instruction from the IBC packet data GmpSolanaPayload, which contains

- data, instruction bytes,
- accounts, account metas, and
- payer\_position, an index at which to inject the relayer's fee payer.

The relayer's payer is inserted as AccountMeta::new(payer, true) (writable + signer) at the attacker-chosen index, and signer privilege propagates through Solana's CPI mechanism. This allows an attacker to craft a GMP packet that executes a CPI to any program with the relayer's fee payer authority, enabling theft of funds (both SOL and SPL tokens) from the relayer.

No restriction exists on which program the packet can target, i.e., which program the CPI will call.

A straightforward exploit targets the system program and transfers the relayer's SOL to the attacker:

1. Attacker sends a GMP packet with receiver = System Program ID.
2. accounts contains a single entry: the attacker's SOL account (writable).
3. payer\_position = 0, placing the relayer as the first account.
4. data encodes a system program Transfer instruction.

The GMP program validates only the attacker's account (trivially matches), then inserts the relayer at index 0. The final CPI becomes `SystemProgram::Transfer { from: RELAYER, to: ATTACKER }`. The relayer signed the outer transaction, so the system program accepts it.

The injected payer is the relayer's single global fee payer keypair. An address configured in `CosmosToSolanaConfig.solana_fee_payer` and reused across all transactions. This means that the relayer's entire operational wallet balance is at risk.

Similarly, the router program's packet callbacks (`recv_packet`, `ack_packet`, and `timeout_packet`) forward relayer-controlled signer authority into registered IBC app CPIs. However, such IBC apps are not arbitrary and must be registered by the permissioned `ID_CUSTOMIZER_ROLE` role.

### Recommendations:

Consider re-architecting the rent payment. For example, replacing the relayer payer injection with the per-user GMP Account PDA as the rent payer. Since it already signs via `invoke_signed`, pre-funding it with SOL allows it to pay rent without exposing the relayer's signer to attacker-controlled CPIs.

**Cosmos Labs:** Resolved with [@151b171433 ...](#). We assume that the relayers are compensated off chain for relaying a particular packet. This might be via a front-end for example. That assumption should mitigate this problem.

**Zenith:** Verified.

## 4.2 High Risk

A total of 2 high risk findings were identified.

### [H-1] Ed25519 pre[]verification PDAs trusted as final truth in Solana Tendermint light client

SEVERITY: High

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

#### Target

- [packages/tendermint-light-client/solana/src/lib.rs](#)
- [programs/ics07-tendermint/src/instructions/pre\\_verify\\_signatures.rs](#)
- [packages/relayer/modules/cosmos-to-solana/src/tx\\_builder/client.rs](#)

#### Description:

SolanaSignatureVerifier treats sig\_verify PDAs as an authoritative oracle for Ed25519 validity: if a PDA at PDA(["sig\_verify", sha256(pk||msg||sig)], program\_id) has is\_valid = 1 at the expected offset, the verifier unconditionally accepts the signature and skips brine\_ed25519 verification.

Those PDAs are created by the ICS07 program's pre\_verify\_signature instruction, which:

- Derives the same signature\_hash,
- Calls the Ed25519 precompile, but only checks bytes inside instruction 0 and ignores the \*\_instruction\_index fields that tell the precompile where it actually read pubkey/msg/signature from.

A malicious relayer can craft a transaction in which:

- The Ed25519 precompile verifies some benign (pk<sub>2</sub>, msg<sub>2</sub>, sig<sub>2</sub>) from another instruction, while
- Instruction 0 contains attacker[]chosen (pk<sub>1</sub>, msg<sub>1</sub>, sig<sub>1</sub>) matching SignatureData.

pre\_verify\_signature will mark (pk<sub>1</sub>, msg<sub>1</sub>, sig<sub>1</sub>) as is\_valid = 1, and SolanaSignatureVerifier will later accept those forged signatures as if they were genuinely precompiled. This allows such a relayer to:

- Make the Tendermint client accept arbitrary fabricated headers/commits,

- Drive the client state (heights, timestamps, app hashes) to attacker-chosen values, and
- Thus forge IBC membership/non-membership proofs and misbehaviour outcomes on Solana.

## Recommendations:

We recommend aligning the producer with the verifier's assumptions:

- Make `pre_verify_signature` either:
  - Enforce `*_instruction_index = u16::MAX` and that the Ed25519 instruction is instruction 0, or
  - Explicitly validate that the same instruction and offsets used by the precompile are the ones being checked for `(pk,msg,sig)`.
- Reject any Ed25519 instruction where these invariants are not met.

**Cosmos Labs:** Resolved with [@86505ac8c6 ...](#)

**Zenith:** Verified.

## [H-2] Relayer-controlled metadata port routes packets to the wrong IBC application, breaking core routing safety

SEVERITY: High

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Medium

### Target

- [recv\\_packet.rs#L31](#)
- [ack\\_packet.rs#L35](#)
- [timeout\\_packet.rs#L33](#)

### Description:

The `ibc_app` PDA, which determines which IBC application receives the packet callback, is derived from the port identifier in `msg.payloads[0]`, the relayer-supplied metadata struct. In the `recv_packet` instruction, this is `msg.payloads[0].dest_port` ([recv\\_packet.rs#L31](#)),

```
30: #[account(  
31:     seeds = [IBCApp::SEED, msg.payloads[0].dest_port.as_bytes()],  
32:     bump  
33: )]  
34: pub ibc_app: Account<'info, IBCApp>,
```

while in `ack_packet` and `timeout_packet` the equivalent field is `msg.payloads[0].source_port`.

When operating in inline mode (the packet carries its payloads directly inside `msg.packet.payloads`, **not** chunked), the handler in `validate_and_reconstruct_packet` ([chunking.rs#L365-368](#)) uses `msg.packet.payloads` as the authoritative payload data. The light client membership proof then verifies the commitment over this inline payload.

Nothing ever checks that the port in `msg.payloads[0]` (metadata) matches the port in `msg.packet.payloads[0]` (verified payload). **The two are entirely independent inputs.**

Exploitability hinges on inline-mode packets paired with metadata entries where `total_chunks = 0`. In this case, the metadata is still consumed for `ibc_app` PDA derivation, but payload/commitment verification uses the inline payload.

A relayer with `RELAYER_ROLE` can exploit this gap by submitting a legitimate inline packet, for example, a token transfer with `dest_port = "transfer"`, while setting the metadata

`dest_port` to a different registered application, such as "oracle".

The light client verifies the actual transfer packet, but the `on_recv_packet` CPI goes to the oracle app instead of the transfer app. The receipt and acknowledgement are written against the verified packet, so the transfer app never receives its callback.

The vulnerability exists across all three packet lifecycle instructions. Following is the concrete impact on IFT (token transfers) and GMP, which together handle cross-chain value movement on Solana:

- *recv\_packet*: IFT token minting on the destination chain happens via GMP's `on_recv_packet`, which executes a CPI to `ift_mint` ([on\\_recv\\_packet.rs#L198](#)). If the router sends this callback to the wrong app and that app returns a valid acknowledgement, the packet receipt is written permanently and the source chain sees a success ack. The correct app (GMP) never executes `ift_mint`, so no tokens are minted on the destination. The source chain considers the transfer complete, resulting in a cross-chain token loss.
- *ack\_packet*: GMP's `on_ack_packet` creates a `GMPCallResultAccount` ([on\\_ack\\_packet.rs#L64](#)), which IFT's `finalize_transfer` reads to decide whether to refund or complete ([finalize\\_transfer.rs#L153](#)). If the ack callback goes to the wrong app, `packet_commitment` is closed (`close = relayer`) but `GMPCallResultAccount` is never created. `finalize_transfer` becomes permanently uncallable. Consequently, the `PendingTransfer` PDA stays alive, the sender's burned tokens are never refunded, and the transfer can never complete.
- *timeout\_packet*: GMP's `on_timeout_packet` creates a `GMPCallResultAccount` with `CallResultStatus::Timeout` ([on\\_timeout\\_packet.rs#L64](#)), which triggers the refund path in `finalize_transfer` ([finalize\\_transfer.rs#L154-175](#)). If misrouted, the commitment is closed but no `GMPCallResultAccount` exists. The sender's tokens were burned in `ift_transfer` and no refund is possible.

## Recommendations:

Consider cross-validating the metadata port against the inline payload port before or during account resolution.

For `recv_packet`, require that `msg.payloads[0].dest_port == msg.packet.payloads[0].dest_port` when inline payloads are present, and apply the equivalent check for `source_port` in `ack_packet` and `timeout_packet`.

Alternatively, derive the `ibc_app` PDA directly from the inline payload's port field when the packet carries inline payloads, removing the metadata dependency from the routing decision entirely.

**Cosmos Labs:** Resolved with [@b66d7de3f4 ...](#)

**Zenith:** Verified.

## 4.3 Medium Risk

A total of 7 medium risk findings were identified.

### [M-1] Shared `ClientSequence` counter causes stale PDA derivation and packet send DoS

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

#### Target

- [programs/solana/programs/ics26-router/src/instructions/send\\_packet.rs#L108-L121](#)
- [programs/solana/programs/ics26-router/src/instructions/send\\_packet.rs#L174-L177](#)

#### Description

The `send_packet` instruction handler reads `ClientSequence.next_sequence_send` at runtime to derive the expected packet commitment PDA, then checks it against the account the caller provided in the transaction's account list. On Solana, account addresses are fixed at signing time, so the caller must read the counter off-chain and derive the PDA before submitting.

If any other sender on the same `source_client` lands first and causes the sequence counter to be incremented, the runtime computes a different PDA than what the caller supplied. The equality check in

```
166: let (expected_pda, bump) = Pubkey::find_program_address(  
167:     &[  
168:         Commitment::PACKET_COMMITMENT_SEED,  
169:         source_client.as_bytes(),  
170:         &sequence.to_le_bytes(),  
171:     ],  
172:     &crate::ID,  
173: );  
174: require!(  
175:     packet_commitment_info.key() == expected_pda,  
176:     RouterError::InvalidChunkAccount  
177: );
```

fails with `InvalidChunkAccount`, reverting the transaction.

Unlike most Solana programs where mutable state is scoped per-user or per-pool, here a single `ClientSequence` account is shared across all IBC applications and all senders on a client. Under organic high throughput, multiple legitimate senders will frequently invalidate each other's pre-derived PDAs without any malicious intent. This also opens the door to targeted griefing via transaction ordering.

## Recommendations

Consider a per-sender sequence counter.

**Cosmos Labs:** Resolved with [@a10d837ce5 ...](#). Apps should be able to either receive a sequence number from the original caller or hash some salt to produce a random sequence that is very likely to be unused. The frontends could also check whether a packet commitment account exists before sending a sequence. Off-chain coordination is an accepted risk.

**Zenith:** Verified.

## [M-2] Pre-funded packet commitment PDA blocks `send_packet` for targeted senders

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

### Target

- [ics26-router/src/instructions/send\\_packet.rs#L191-L203](#)

### Description

The `send_packet` instruction creates packet commitment PDAs at runtime using `anchor_lang::system_program::create_account` via CPI to the System Program. This manual path is necessary because the sequence number is computed at runtime through `calculate_namespaced_sequence` and cannot be expressed in Anchor's static seed derivation.

The System Program's `CreateAccount` instruction contains a hard precondition: the destination account must have zero lamports. If any lamports are already present, the instruction fails with `AccountAlreadyInUse`. Since Solana allows transferring lamports to any address (including off-curve PDA addresses) without requiring a signature from the destination, an adversary can pre-fund the target PDA to trigger this failure.

All seed components for the packet commitment PDA are publicly derivable:

- `PACKET_COMMITMENT_SEED` is a constant.
- `source_client` is the IBC client identifier, visible on-chain.
- `sequence` is computed as  $\text{base\_sequence} * 10000 + \text{sha256}(\text{app\_program} || \text{payer}) \% 10000$ , where `base_sequence` is readable from the `ClientSequence` account's `next_sequence_send` field, and `app_program` / `payer` are public keys.

An attacker reads `next_sequence_send` on-chain, computes the target sequence for a known sender, derives the PDA, and transfers 1 lamport to it. When the legitimate sender's `send_packet` transaction executes, the `create_account` CPI at [line 191](#) fails, reverting the entire instruction. Because the revert is atomic, `next_sequence_send` is never incremented. The same sender retries with the same sequence and hits the same pre-funded PDA indefinitely.

The targeted sender remains blocked until a different sender (who produces a different sequence suffix) successfully advances the base sequence.

## Recommendations

Consider replacing the raw `create_account` CPI with the defensive pattern used internally by Anchor's `init`: when the PDA already holds lamports, transfer only the delta needed for rent exemption, then use `allocate` and `assign` via `invoke_signed` to initialize the account. This provides the same pre-funding resistance as `init_if_needed` while keeping the runtime-computed seed flexibility the manual path needs.

**Cosmos Labs:** Resolved with [@a10d837ce5 ...](#)

**Zenith:** Verified.

## [M-3] IFT refund mints are subject to rate limit and pause state, contrary to spec

SEVERITY: Medium

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Medium

### Target

- [programs/solana/programs/ift/src/instructions/finalize\\_transfer.rs#L30-L38](#)
- [programs/solana/programs/ift/src/helpers.rs#L15-L42](#)

### Description

The IFT architecture ADR ([docs/adr/solana-ift-architecture.md#L214](#)) states:

"All mint paths (except refunds) respect daily rate limits and pause state."

The implementation diverges from this intent in two ways:

1. **Rate limit:** `finalize_transfer()` issues refund mints through `mint_to_account()`, which unconditionally calls `check_and_update_mint_rate_limit()`. Refunds on timeout and error ACK therefore consume daily budget just like inbound mints, while only success ACK frees budget via `reduce_mint_rate_limit_usage()`.  
This asymmetry could be exploited to grief the refund mechanism: an attacker burns tokens via `ift_transfer()` with a short timeout (as reported in another finding, the minimum timeout is only 12 seconds, instead of the intended 60 seconds), lets the packet time out, and collects a refund mint. The net token balance remains unchanged, but the rate-limit budget is reduced. Repeating this exhausts the daily limit at the cost of transaction fees only. Once exhausted, legitimate `ift_mint` calls and other users' refunds revert until the next day.
2. **Pause state:** Checking `constraint = !app_state.paused @ IFTErrors::AppPaused (`[finalize\\_transfer.rs#L35](#)`)` blocks all refunds while the token bridge is paused.

### Recommendations

Consider skipping the pause check and rate limit on the timeout and error-ACK refund paths in `finalize_transfer()`. Both controls remain appropriate in the other scenarios.

**Cosmos Labs:** Acknowledged. We will not address this. This is an acceptable risk to us at this point. In the future, we want to mitigate this with a more sophisticated rate-limiting logic.

## [M-4] Hardcoded GMP encoding constant prevents Solana-to-EVM IFT transfers

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: High

### Target

- [programs/solana/programs/ics27-gmp/src/constants.rs#L14](#)
- [programs/solana/programs/ics27-gmp/src/instructions/send\\_call.rs#L155](#)
- [programs/solana/programs/ift/src/instructions/ift\\_transfer.rs#L249-L261](#)

### Description:

ChainOptions::Evm in the IFT program produces a correct ABI-encoded iftMint(address, uint256) inner payload via construct\_evm\_mint\_call(). The GMP layer, however, hardcodes ICS27\_ENCODING = "application/x-protobuf" and protobuf-encodes the GmpPacketData wrapper for every outbound packet regardless of the destination.

On the EVM side, the onRecvPacket() function [rejects any encoding other than "application/x-solidity-abi"](#). abi.decode() would revert on the protobuf-encoded wrapper.

The ChainOptions::Evm path is therefore broken. Tokens are burned on Solana, the packet is committed, but the EVM destination always rejects it and reverts. The ICS26 EVM router [catches the revert](#) and writes an error-ACK, so the sender on Solana can reclaim tokens via finalize\_transfer once the error ACK is relayed back. As a result, the IFT transfers from Solana to EVM are effectively non-functional.

### Recommendations:

Consider selecting the correct encoding in GMP's send\_call based on the destination chain.

**Cosmos Labs:** Resolved with [@2b61dcd182...](#)

**Zenith:** Verified.

## [M-5] PendingTransfer PDA creation can be grieved via lamport prefunding

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

### Target

- [programs/solana/programs/ift/src/instructions/ift\\_transfer.rs#L360-L374](#)

### Description:

`create_pending_transfer_account` initializes the PendingTransfer PDA using `system_instruction::create_account`. On Solana, this instruction fails with `AccountAlreadyInUse` if the destination account already holds any lamports, even if it has no data and is still owned by the System Program.

An attacker can derive the PDA, and send a dust lamport amount to it before the `ift_transfer` instruction lands, thereby grieving the victim's cross-chain IFT transfer due to the `create_account` erroring out.

This is the same class of issue as the `packet_commitment` PDA prefunding vulnerability in `send_packet.rs`, reported in "Pre-funded packet commitment PDA blocks `send_packet` for targeted senders".

### Recommendations:

Consider replacing the raw `create_account` CPI with the defensive pattern used internally by Anchor's `init`: when the PDA already holds lamports, transfer only the delta needed for rent exemption, then use `allocate` and `assign` via `invoke_signed` to initialize the account. This provides the same pre-funding resistance as `init_if_needed` while keeping the runtime-computed seed flexibility the manual path needs.

**Cosmos Labs:** Resolved with [PR-1006](#).

**Zenith:** Verified.

## [M-6] Anchor's Borsh-wrapped return data corrupts acknowledgement bytes delivered to source chain IBC apps

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: High

### Target

- [programs/solana/packages/solana-ibc-types/src/ibc\\_app.rs#L158-L162](#)
- [programs/solana/packages/ics26-router/src/instructions/recvd\\_packet.rs#L222-L236](#)

### Description:

Anchor's #[program] codegen [Borsh-serializes the return value](#) of every instruction handler before calling `set_return_data()`. For `Result<Vec, this prepends a 4-byte little-endian length prefix. A 7-byte ack becomes 11 bytes: 07 00 00 00 followed by the original 7 bytes.`

The router's CPI helper in `ibc_app.rs:160` reads the return data as-is without stripping this Borsh envelope:

```
158: // Get acknowledgement from return data
159: match get_return_data() {
160:     Some((program_id, data)) if program_id == *ctx.program.key =>
        Ok(data),
161:     _ => err!(IbcAppError::InvalidAppResponse),
162: }
```

The wrapped bytes propagate through the full ack pipeline: committed as `sha256(0x02 || sha256(borsh_wrapped_ack))`, emitted in `WriteAcknowledgementEvent`, [forwarded by the relayer](#), and delivered to the source chain's `onAcknowledgementPacket`. Commitment verification still passes because both sides hash the same wrapped bytes, but the source chain app receives `[4_byte_len || actual_ack]` instead of `actual_ack`.

This breaks the [Solidity GMP app's error detection](#),

```
171: IBCSenderCallbacksLib.ackPacketCallback(
172:     sender, keccak256(msg_.acknowledgement) !=
        ICS24Host.KECCAK256_UNIVERSAL_ERROR_ACK, msg_
173: );
```

The Borsh-wrapped ack never matches the precomputed hash of the raw error constant, so error ACKs are always misclassified as success. Any app that ABI- or protobuf-decodes the ack content will also fail or produce garbage.

For comparison, the EVM side does not have this problem.

[The onRecvPacket function returns raw bytes memory with no wrapping.](#)

### Recommendations:

Consider Borsh-deserializing the return data in `on_recv_packet` to strip the length prefix before returning it to the router.

**Cosmos Labs:** Resolved with [@f63fb73681...](#) and [@80f10aaf0b...](#)

**Zenith:** Verified.

## [M-7] No pause/unpause mechanism for router despite documented emergency roles

SEVERITY: Medium

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [ics26-router/src/instructions/recv\\_packet.rs](#)

### Description:

The RouterState account has no paused flag and the program exposes no pause or unpause instruction, despite the project's own documentation defining PAUSER\_ROLE (ID 2) and UNPAUSER\_ROLE (ID 3) for exactly this purpose.

In the event of a security incident, such as a light client compromise, a malicious IBC app, or an exploit in the counterparty chain, there is no atomic mechanism to halt all IBC packet flow.

The Ethereum implementation of ICS26Router includes a pause/unpause mechanism gated by the corresponding roles, making this a deviation from the cross-chain specification and a reduction in operational safety guarantees on the Solana side.

### Recommendations:

We recommend adding a `paused: bool` field to the RouterState account and implementing two new instructions, `pause` gated by PAUSER\_ROLE and `unpause` gated by UNPAUSER\_ROLE, that toggle this flag. Each packet-processing instruction (`send_packet`, `recv_packet`, `ack_packet`, `timeout_packet`) should check `require(!router_state.paused, RouterError::ProtocolPaused)` at the top of its handler, before any state mutation. This provides a single-transaction emergency brake that immediately halts all IBC traffic regardless of the number of registered clients.

**Cosmos Labs:** Resolved with [@bd7cc43999 ...](#)

**Zenith:** Verified.

## 4.4 Low Risk

A total of 17 low risk findings were identified.

### [L-1] Insufficient port ID validation in add\_ibc\_app

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

#### Target

- [ics26-router/src/instructions/add\\_ibc\\_app.rs](#)

#### Description:

The `add_ibc_app` instruction validates the `port_id` parameter with only a single check, `require(!port_id.is_empty(), RouterError::InvalidPortIdentifier)`, despite the codebase defining `MIN_PORT_ID_LENGTH` (2) and `MAX_PORT_ID_LENGTH` (128) in `state.rs` that are never enforced.

This means a privileged user with `ID_CUSTOMIZER_ROLE` can register a port with a single-character ID, an ID containing arbitrary bytes (including non-printable or special characters). This stands in contrast to `add_client`, which performs thorough character-set validation, length bounds checking, and reserved-prefix rejection via `validate_custom_ibc_identifier`.

An excessively long port ID could cause serialization issues when the `IBCApp` account is written, since Anchor's `max_len` only governs space allocation, not input validation. Non-standard characters could cause interoperability problems with counterparty chains that enforce ICS-24 identifier rules, and single-character IDs weaken the namespace's resistance to accidental collisions.

#### Recommendations:

We recommend applying the same validation used for client IDs: enforce that `port_id.len() ≥ MIN_PORT_ID_LENGTH` & `port_id.len() ≤ MAX_PORT_ID_LENGTH`, and validate that the port ID contains only permitted characters (alphanumeric plus `._+-#[]<>`). Alternatively, reuse or adapt `validate_custom_ibc_identifier` from `client.rs` so that both client IDs and port IDs share a single, consistent validation function aligned with ICS-24 identifier requirements.

**Cosmos Labs:** Resolved with [@9af7826223 ...](#)

**Zenith:** Verified.

## [L-2] Initialization of Solana programs can be frontrun to set an arbitrary admin

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [access-manager/src/instructions/initialize.rs#L29](#)
- [ift/src/instructions/initialize.rs#L28](#)
- [ics26-router/src/instructions/initialize.rs#L22](#)
- [ics07-tendermint/src/instructions/initialize.rs#L39](#)
- [attestation/src/instructions/initialize.rs#L31](#)
- [ics27-gmp/src/instructions/initialize.rs#L25](#)

### Description:

Every Solana program in the repository follows the same initialization pattern: the `initialize` instruction accepts an `admin` (or `access_manager` / `authority`) as an instruction argument, creates a PDA with fixed seeds via Anchor's `init` constraint, and stores the provided value as the privileged role. The only signer required is `payer`, which can be any account.

For example, in the `access-manager` program, the `Initialize` account struct requires only a `payer: Signer` and takes `admin: Pubkey` as instruction data. The handler then calls `access_manager.grant_role(roles::ADMIN_ROLE, admin)` with whatever pubkey was passed. The same pattern applies to `ift` (sets `app_state.admin`), `ics26-router` (sets `router_state.access_manager`), `ics07-tendermint`, `attestation`, and `ics27-gmp`.

Since the PDA addresses are deterministic (fixed seeds), an attacker could front-run the legitimate `initialize` transaction and submit their own address as the `admin`. Anchor's `init` constraint guarantees only the first call succeeds. Subsequent attempts fail because the PDA already exists. The legitimate deployer's transaction would then revert, leaving the attacker in control of the program.

### Recommendations:

Consider requiring the program's upgrade authority to be a signer in the `Initialize` accounts struct.

**Cosmos Labs:** Resolved with [@8c438f2f91 ...](#)

**Zenith:** Verified.

### [L-3] Reserved role IDs are documented as unimplemented but remain fully assignable

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

#### Target

- [programs/access-manager/src/instructions/grant\\_role.rs](#)

#### Description:

The documentation states that role IDs 4, 5, and 7 are reserved and “not currently implemented on Solana,” but the on-chain logic does not enforce this restriction.

In practice, `grant_role` and `revoke_role` only block `PUBLIC_ROLE` and allow arbitrary non-public `u64` role IDs, including reserved IDs. This creates a specification/implementation mismatch and may lead to governance misconfiguration, especially if future upgrades later activate semantics for those reserved IDs.

#### Recommendations:

We recommend enforcing an explicit allowlist/denylist of role IDs on-chain so reserved IDs cannot be granted or revoked until officially activated. We also recommend aligning documentation and program behavior, and adding tests that verify reserved role IDs are rejected while active role IDs remain supported.

**Cosmos Labs:** Resolved with [@e6afcbdab0 ...](#)

**Zenith:** Verified.

## [L-4] Missing grantee address validation allows assigning roles to zero address

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [programs/access-manager/src/instructions/grant\\_role.rs](#)

### Description:

The `grant_role` instruction of the `AccessManager` program validates only `role_id` and does not validate the `account` argument.

As a result, an admin can grant roles to `Pubkey::default()` (zero address), which is not a meaningful operator key.

### Recommendations:

We recommend adding explicit input validation in `grant_role` to reject `Pubkey::default()` (zero address) as a grantee and returning a dedicated error (e.g., `InvalidAccount`).

**Cosmos Labs:** Resolved with [@988904299a...](#)

**Zenith:** Verified.

## [L-5] Missing admin sanity check can permanently brick governance

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [programs/access-manager/src/instructions/initialize.rs](#)

### Description:

The `initialize` instruction of the `AccessManager` accepts arbitrary `admin: Pubkey` without validating that it is non-default and controllable. If `Pubkey::default()` (or an unintended key) is provided, no valid signer may control `ADMIN_ROLE`, effectively locking governance/admin operations from genesis.

### Recommendations:

We recommend rejecting `Pubkey::default()` explicitly and preferring signer-based admin assignment (`admin.key()`) over free-form pubkey input.

**Cosmos Labs:** Resolved with [@988904299a ...](#)

**Zenith:** Verified.

## [L-6] Removing/deactivating bridge can permanently block pending-transfer finalization

SEVERITY: Low

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [programs/ift/src/instructions/remove\\_ift\\_bridge.rs](#)

### Description:

The `finalize_transfer` instruction requires an active `IFTBridge` for the `(mint, client_id)` pair: it enforces `constraint = ift_bridge.active @ IFTErrors::BridgeNotActive` and derives the bridge PDA from `(IFT_BRIDGE_SEED, mint, client_id)`.

The `remove_ift_bridge` instruction closes this bridge account (and returns rent to payer) without checking for existing `PendingTransfer` PDAs. Once a bridge is removed or deactivated, any in-flight `PendingTransfer` for that `(mint, client_id)` can no longer be finalized, because callers cannot satisfy the `ift_bridge` constraint required by `finalize_transfer`. Even if a valid `GMPCallResultAccount` exists (timeout or ack already processed by GMP), there is no way to complete or refund those transfers, effectively stranding the burned tokens tied to those pending transfers.

### Recommendations:

We recommend decoupling pending-transfer finalization from the bridge's removal lifecycle.

**Cosmos Labs:** Resolved with [@c7bd86e4ba ...](#)

**Zenith:** Verified.

## [L-7] Admin can brick minting but app state stays “live”

SEVERITY: Low

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [programs/ift/src/instructions/admin.rs](#)

### Description:

The `revoke_mint_authority` instruction transfers the SPL mint's `MintTokens` authority from the IFT mint-authority PDA to an arbitrary `new_mint_authority`, but it does not close or otherwise disable the associated `IFTAppState` / `IFTAppMintState` or bridges.

After this call, all IFT mint flows (`ift_mint`, `admin_mint`, and refund mints in `finalize_transfer`) still execute their logic and reach the SPL/Token `2022 mint_to CPI`, but the token program will reject them because the PDA is no longer an authorized mint authority.

Outbound transfers can still burn tokens and send GMP calls, but inbound mints and refunds will fail at the token level, effectively bricking minting for that mint while leaving the app state and bridges appearing “live”.

This behavior contradicts the ADR, which describes `revoke_mint_authority` as reclaiming authority and “closing app state”, and can surprise operators or integrators who believe the system is either fully active or fully decommissioned.

### Recommendations:

We recommend treating `revoke_mint_authority` as a formal shutdown and explicitly closing the per-mint IFT state (and optionally deactivating related bridges) so clients can detect that the mint is no longer usable, or

**Cosmos Labs:** Resolved with [@f3530a1b8a ...](#)

**Zenith:** Verified.

## [L-8] No explicit max-account bound in GmpSolanaPayload

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

### Target

- [solana-ibc-proto/src/lib.rs](#)

### Description:

The `GmpSolanaPayload` deserialization logic does not enforce any explicit upper bound on the number of accounts in `raw.accounts`, even though `GmpValidationError` defines a `TooManyAccounts` (max 32) error.

As a result, `GmpSolanaPayload::try_from` will accept and materialize arbitrarily large account lists, and `on_recv_packet` will then perform  $O(N)$  validation and CPI account construction over all of them.

The lack of a hard bound both contradicts the intended “max 32 accounts” constraint and increases the attack surface for compute-heavy or bloated packets that aim to maximize per-packet resource usage.

### Recommendations:

We recommend enforcing a strict maximum account count when converting `RawGmpSolanaPayload` to `GmpSolanaPayload` (e.g. returning `GmpValidationError::TooManyAccounts` if `raw.accounts.len() > 32`), aligning this bound with ICS-27 GMP and relayer expectations.

**Cosmos Labs:** Resolved with [@41818c6ea0 ...](#)

**Zenith:** Verified.

## [L-9] Missing validation of access\_manager pubkey during initialization

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [programs/ics26-router/src/instructions/initialize.rs](#)

### Description:

The `initialize` instruction accepts an arbitrary `access_manager: Pubkey` and writes it into `router_state.access_manager` without checking that this pubkey actually corresponds to a valid address and an initialized `access_manager` deployment.

Since all subsequent role checks and CPI protections derive the `access_manager PDA` via `seeds::program = router_state.access_manager`, misconfiguring this value will cause the router's authorization model to fail in non-obvious ways.

### Recommendations:

We recommend adding explicit validation in `initialize` to ensure that is not the default pubkey.

**Cosmos Labs:** Resolved with [@fa913e302e ...](#)

**Zenith:** Verified.

## [L-10] Single-step admin transfer

SEVERITY: Low

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [admin.rs](#)

### Description:

The `set_admin` function in `admin.rs` (`pub fn set_admin(ctx: Context<SetAdmin>, new_admin: Pubkey) → Result<()> { ... }`) performs admin transfer as a single atomic operation, directly assigning `app_state.admin = new_admin` based solely on the current admin's signature. If the current admin supplies an incorrect or unintended `new_admin` pubkey (e.g., typo, wrong address, or lost key), control is permanently and irrecoverably transferred to an unusable or unknown account, with no built-in recovery mechanism.

### Recommendations:

We recommend replacing the single-step `set_admin` with a two-step handover flow, such as `propose_admin(new_admin)` (callable by the current admin, storing a pending admin) followed by `accept_admin()` (callable and signed by the proposed new admin), ensuring that control is only transferred to an address that can actually sign and that typos or incorrect keys do not permanently lock the system.

**Cosmos Labs:** Resolved with [@3b594aab7d ...](#)

**Zenith:** Verified.

## [L-11] MIN\_TIMEOUT\_DURATION constant defined but never enforced in IFT transfer validation

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [programs/solana/programs/ift/src/constants.rs#L16](#)
- [programs/solana/programs/ift/src/instructions/ift\\_transfer.rs#L145-L157](#)

### Description

The IFT program defines MIN\_TIMEOUT\_DURATION as 60 seconds in [constants.rs#L16](#), but the timeout validation in `ift_transfer()` [only checks that the caller-supplied timeout is in the future](#) (`> clock.unix_timestamp`) and within MAX\_TIMEOUT\_DURATION. It never checks against MIN\_TIMEOUT\_DURATION.

Therefore, a user can set `timeout_timestamp` to `current_time + 1`. However, the GMP layer does enforce its own minimum of 12 seconds at [send\\_call.rs#L129](#), but this is well below the 60-second floor the IFT constant was designed to impose.

### Recommendations

Consider checking that the caller-supplied timeout is at least MIN\_TIMEOUT\_DURATION in the future.

**Cosmos Labs:** Resolved with [@ecdf2b76e5 ...](#)

**Zenith:** Verified.

## [L-12] Privileged ID\_CUSTOMIZER\_ROLE role can permanently block client registration through PDA seed collision

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [programs/solana/packages/solana-ibc-types/src/router.rs#L244](#)
- [programs/solana/packages/solana-ibc-types/src/router.rs#L255](#)
- [programs/solana/programs/ics26-router/src/instructions/client.rs#L30](#)
- [programs/solana/programs/ics26-router/src/instructions/client.rs#L39](#)

### Description:

The `Client` and `ClientSequence` PDA types in the ICS-26 router use seeds that share a byte-prefix. `**Cosmos Labs:**:SEED` is `b"client"`, while `ClientSequence::SEED` is `b"client_sequence"`.

Solana's `find_program_address` concatenates all seed elements into a single byte buffer before hashing, without inserting any length delimiter between them. Both PDA derivations append `client_id.as_bytes()` after the seed constant:

- `Client` PDA: `hash("client" || client_id)`
- `ClientSequence` PDA: `hash("client_sequence" || client_id)`

If a `client_id` is set to `"_sequence" + X`, the `Client` PDA concatenation becomes `"client" || "_sequenceX" = "client_sequenceX"`, which is byte-identical to the `ClientSequence` PDA for client X: `"client_sequence" || "X" = "client_sequenceX"`. Both PDAs resolve to the same on-chain address.

The `validate_custom_ibc_identifier` function [only blocks IDs](#) starting with `"client-"` or `"channel-"`. It does not block the `"_sequence"` prefix, and underscores are among the allowed characters, so a client ID like `"_sequenceABCD"` passes all validation checks.

A holder of `ID_CUSTOMIZER_ROLE` can exploit this as follows:

1. Call `add_client("_sequenceXYZ", ...)`. Anchor initializes the `Client` PDA at address `A = find_program_address(["client", "_sequenceXYZ"])` and a separate `ClientSequence` PDA at a different address (since `["client_sequence", "_sequenceXYZ"]` produces a different hash). Both succeed.

2. When an admin later tries to register client "XYZ", Anchor attempts to init the `ClientSequence` PDA for "XYZ" at `find_program_address(["client_sequence", "XYZ"])`, which equals address A -> already occupied by the attacker's Client PDA from step 1.
3. Anchor's `init` constraint fails with "account already in use", permanently blocking the registration of client "XYZ".

This requires `ID_CUSTOMIZER_ROLE` privileges and only causes a targeted DoS against specific client IDs, with no path to fund theft.

### Recommendations:

Consider adding `"_sequence"` to the blocked prefix list in `validate_custom_ibc_identifier`.

Alternatively, changing `ClientSequence::SEED` to a value that does not start with `"client"` (e.g., `b"cseq"`) would remove the prefix relationship at the root.

**Cosmos Labs:** Resolved with [@04e6c0ab22 ...](#)

**Zenith:** Verified.

## [L-13] IFT mint creation can be grieved via lamport pre-funding of the mint account

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

### Target

- [create\\_and\\_initialize\\_spl\\_token.rs#L170-L181](#)
- [create\\_and\\_initialize\\_spl\\_token.rs#L255-L266](#)

### Description:

Both the `create_token_2022_mint` and `create_legacy_mint` functions allocate the mint account via the raw `system_program::create_account`. The system Program's `CreateAccount` instruction fails with `AccountAlreadyInUse` when the destination already holds any lamports, even with zero data and system program ownership.

By front-running the admin's `create_and_initialize_spl_token` call by transferring a dust amount of lamports to the mint address, the entire instruction can be caused to revert.

Same class of issue as the `PendingTransfer` and packet commitment PDA pre-funding bugs reported elsewhere. Unlike those, the mint is a `Signer` (random keypair) rather than a deterministic PDA, so the address cannot be precomputed from on-chain state alone. The admin can recover by generating a fresh keypair and resubmitting, making the grieving transient rather than persistent.

### Recommendations:

Consider applying the same defensive `create_account` pattern recommended for the PDA-based instances: check whether the account already holds lamports, transfer only the rent-exemption delta, then `allocate` and `assign` via the signer.

**Cosmos Labs:** Resolved with [@e6e9a01695 ...](#)

**Zenith:** Verified.

## [L-14] Lack of attesor set rotation mechanism

SEVERITY: Low

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

### Target

- [attestation/src/instructions/initialize.rs](#)

### Description:

The attestation light client stores a fixed set of trusted Ethereum attesor addresses and a quorum threshold (`min_required_sigs`) in `ClientState` at initialization, but does not expose any instruction to update this attesor set or threshold afterwards.

There is no way to add/remove attestors, change `min_required_sigs`, or rotate compromised keys at runtime; any change requires a full program upgrade with data migration. In a production setting where attestors are ECDSA key holders (EOAs or HSM-backed keys), this creates an operational fragility: compromise or retirement of an attesor key, or the need to adjust the quorum in response to changing participation.

### Recommendations:

We recommend introducing a `update_attestors` instruction, gated by `ADMIN_ROLE` via the existing `access-manager` integration, that allows modifying the attesor address set (add/remove) and updating `min_required_sigs` while preserving safety invariants (e.g., non-empty set,  $\text{threshold} \leq \text{number of attestors}$ , no duplicates).

**Cosmos Labs:** Acknowledged. Will not be addressed because our rotation mechanism is to replace the light client program entirely in `ics26-router` program using the `MigrateClient` instruction. We also have pauser roles in case we need to freeze IBC.

## [L-15] Missing router-side owner check for consensus\_state in send\_packet

SEVERITY: Low

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [send\\_packet.rs](#)

### Description:

The `send_packet` instruction enforces that `client_state` is owned by the `light_client` program but does not enforce any owner constraint on the `consensus_state` account before passing it into the light client's `client_status` entrypoint. This is inconsistent with `recv_packet`, `ack_packet`, and `timeout_packet`, which all require `consensus_state.owner = light_client_program.key()`.

In this codebase, the concrete light clients (ICS07 Tendermint and attestation) enforce both ownership and PDA seeds for `consensus_state` inside their own `client_status` instructions, so a forged or attacker-owned `consensus_state` cannot actually be used; the check is therefore missing only at the router layer and functions as a defense-in-depth gap rather than an exploitable vulnerability under the currently shipped light client implementations.

The risk would become more relevant only if a future or third-party light client exposed a `client_status` instruction that accepts unchecked `AccountInfo` and trusts the passed `consensus_state` without validating owner/seeds.

### Recommendations:

We recommend adding a `consensus_state` owner constraint to `SendPacket` (e.g. `#[account(owner = light_client_program.key()) @ RouterError::InvalidAccountOwner]` pub `consensus_state: AccountInfo<'info>,`) and, optionally, extending `LightClientCpi::invoke_instruction` to assert `consensus_state.owner = client.client_program_id`. This aligns `send_packet` with the other packet-handling instructions, hardens the router against misconfigured or less-rigorous light clients, and makes the security assumptions explicit at the router boundary rather than relying solely on downstream light client implementations.

**Cosmos Labs:** Resolved with [@0a22ff4c79...](#)

**Zenith:** Verified.

## [L-16] `verify_membership` does not reject zero-commitment attestations, blurring the boundary with non-membership proofs

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [attestation/src/instructions/verify\\_membership.rs#L64-78](#)
- [attestation/src/instructions/verify\\_non\\_membership.rs#L67-73](#)

### Description:

In the attestation-based light client, both `verify_membership` and `verify_non_membership` operate on `PacketAttestation` entries containing `PacketCompact` structs with a `path` and `commitment` field. The convention for distinguishing presence from absence is the commitment value:

`verify_non_membership` requires `packet.commitment == [0u8; 32]` at [line 73](#), treating a zeroed commitment as the sentinel for "value absent."

However, `verify_membership` does not perform such a guard. It matches the caller-supplied `value_hash` against `packet.commitment` at [lines 76-78](#) without ever checking that the commitment is non-zero. If a caller were to submit `value = [0u8; 32]`, the same attestation entry with a zeroed commitment would pass both `verify_membership` and `verify_non_membership` for the same path and height.

In practice, this is currently not exploitable: all IBC packet commitments are SHA-256 or keccak256 digests, so producing a legitimate zero commitment is computationally infeasible. The `require(!msg.value.is_empty())` check on [line 29](#) also prevents an empty value, but does not prevent a 32-byte zero value from being passed in.

Membership and non-membership verification are supposed to be mutually exclusive. A path should satisfy exactly one of them at any given height. Without an explicit zero-commitment rejection in `verify_membership`, this mutual exclusion depends on the hash function rather than being enforced structurally. Future changes to commitment schemes or protocol extensions could break this assumption.

**Recommendations:**

Consider adding `require!(packet.commitment != [0u8; 32], ErrorCode::CommitmentMismatch)` in `verify_membership` before the value comparison. This enforces the membership/non-membership boundary at the code level and has no effect on legitimate operations since real commitments are never zero.

**Cosmos Labs:** Resolved with [@34a2d04165 ...](#)

**Zenith:** Verified.

## [L-17] Attestation light client status never expires, lacking a liveness check on stale consensus states

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Low

### Target

- [attestation/src/instructions/client\\_status.rs#L21-29](#)

### Description:

The `ClientStatus` enum defined in the ICS-25 handler includes an `Expired` variant alongside `Active` and `Frozen`. The `ics07-tendermint` light client implements expiry detection by comparing the consensus state's timestamp against the on-chain clock and the configured `trusting_period`, returning `Expired` when the elapsed time exceeds the trusting period.

The attestation light client only checks `is_frozen` in its `client_status` function.

```
21: pub fn client_status(ctx: Context<ClientStatus>) → Result<()> {
22:     let status = if ctx.accounts.client_state.is_frozen {
23:         ics25_handler::ClientStatus::Frozen
24:     } else {
25:         ics25_handler::ClientStatus::Active
26:     };
27:     set_return_data(&[status.into()]);
28:     Ok(())
29: }
```

If the client is not frozen, it unconditionally returns `Active`. There is no timestamp comparison, no trusting period field on its `ClientState`, and no path that returns `Expired`. A consensus state created months or years ago remains valid indefinitely.

Note that the attestation client has a different trust model than the Tendermint client. The attester set is stored in `ClientState.attestor_addresses` and checked at verification time against the current on-chain list. If an attester key is compromised, the admin can remove it, and all past attestations using that key stop verifying immediately. A compromised attester that is still in the trusted set can still sign fresh attestations at arbitrary heights, so expiring old consensus states does not meaningfully mitigate key compromise.

That said, a trusting period would serve as a liveness canary: if the client hasn't received an

update in a long time, something is likely wrong, and it is safer to halt packet operations than to continue operating against potentially stale state.

### Recommendations:

Consider adding a `trusting_period` parameter to the attestation `ClientState` as a defense-in-depth measure. This would halt packet operations during prolonged inactivity rather than allowing the client to remain active indefinitely without updates.

**Cosmos Labs:** Acknowledged. We decided to leave this as an accepted risk since the trusting period doesn't serve the same security benefits as it does in the Tendermint light client. Attestors are trusted, and this sanity check doesn't meaningfully impact any specific risk scenario.

## 4.5 Informational

A total of 10 informational findings were identified.

### [I-1] Program whitelist accepts duplicate entries, reducing effective capacity

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

#### Target

- [access-manager/src/instructions/set\\_whitelisted\\_programs.rs#L23-38](#)

#### Description:

The `set_whitelisted_programs` instruction of the `access-manager` program stores the provided `whitelisted_programs: Vec<Pubkey>` directly into `AccessManager.whitelisted_programs` without checking for or removing duplicate entries. The whitelist is capped at 8 entries via Anchor's `#[max_len(8)]` constraint on the [state definition](#).

With only 8 available slots, duplicate entries reduce the number of distinct programs that can be whitelisted. For example, an admin submitting `[A, A, B, B, C, C, D, D]` would fill all 8 slots while only whitelisting 4 unique programs.

#### Recommendations:

Consider deduplicating the input vector before storing it, or rejecting the transaction if duplicates are detected.

**Cosmos Labs:** Resolved with [@cb971742cf ...](#)

**Zenith:** Verified.

## [I-2] Unused ProgramExtendedEvent event

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [access-manager/src/events.rs#L35-42](#)

### Description:

The access-manager program defines five events in `events.rs`. Four of these are imported and emitted by their corresponding instruction handlers:

- `RoleGrantedEvent`
- `RoleRevokedEvent`
- `ProgramUpgradedEvent`
- `WhitelistedProgramsUpdatedEvent`

However, `ProgramExtendedEvent` is never imported, emitted, or referenced anywhere outside its definition. This suggests that it may be a remnant of an unimplemented feature or an oversight during development.

### Recommendations:

Consider removing the `ProgramExtendedEvent` struct.

**Cosmos Labs:** Resolved with [@7547f10247 ...](#)

**Zenith:** Verified.

## [I-3] Documentation inconsistency regarding CPI restrictions in upgrade\_program instruction

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [programs/access-manager/src/instructions/upgrade\\_program.rs](#)
- [programs/solana/README.md](#)

### Description:

The documentation states that Cross-Program Invocation (CPI) calls to the upgrade\_program instruction are blocked. However, the current implementation authorizes the instruction using require\_admin, which allows execution both through direct calls and CPI calls from whitelisted programs.

This creates a discrepancy between the documented behavior and the actual implementation. While this does not introduce a direct security vulnerability or enable unauthorized upgrades, it may mislead developers and auditors regarding the effective access control model of the upgrade\_program instruction.

Such inconsistencies can result in incorrect assumptions about the system's security guarantees and may lead to improper integration decisions by downstream programs interacting with the AccessManager.

### Recommendations:

We recommend updating the documentation to accurately reflect the current behavior of the upgrade\_program instruction, explicitly stating that CPI calls from whitelisted programs are permitted via the require\_admin authorization mechanism.

**Cosmos Labs:** Resolved with [@06999ca2c5 ...](#)

**Zenith:** Verified.

## [I-4] Reduce MAX\_CLIENT\_ID\_LENGTH to avoid exceeding Solana's PDA seed limit

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [programs/solana/packages/solana-ibc-proto/src/lib.rs#L41](#)
- [programs/solana/programs/ics26-router/src/instructions/client.rs#L30](#)
- [programs/solana/programs/ics26-router/src/instructions/client.rs#L206](#)

### Description:

MAX\_CLIENT\_ID\_LENGTH is defined as 64 bytes and feeds into both the `ClientId` constrained type (`ConstrainedString<1, 64>`) and every `#[max_len(64)]` annotation on client ID account fields. However, every PDA derivation across the router, GMP, and IFT programs passes the raw client ID as a single seed via `client_id.as_bytes()`. Solana enforces a hard 32-byte per-seed limit (`MAX_SEED_LEN`), meaning any client ID longer than 32 bytes will cause `Pubkey::create_program_address` to fail with `MaxSeedLengthExceeded`.

This creates a dead validation range: IDs between 33 and 64 bytes pass the `ConstrainedString` check but cannot produce a valid PDA. The router's own `AddClient` instruction uses `seeds = [**Cosmos Labs**:SEED, client_id.as_bytes()]`, so such a client can never be registered in the first place. The `validate_custom_ibc_identifier` function widens the gap further by allowing IDs up to 128 bytes.

Users submitting 33-64 byte IDs get opaque `ConstraintSeeds` errors instead of clear validation failures, and the `ClientId` type misleadingly signals that 64-byte identifiers are supported.

### Recommendations:

Consider reducing `MAX_CLIENT_ID_LENGTH` to 32 to match Solana's `MAX_SEED_LEN`. The upper bound in `validate_custom_ibc_identifier` should be adjusted accordingly.

**Cosmos Labs:** Resolved with [@7295252d0c ...](#)

**Zenith:** Verified.

## [I-5] Packet reconstruction `validate_and_reconstruct_packet` function accepts empty payloads due to NAND check instead of XOR

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [programs/solana/programs/ics26-router/src/utils/chunking.rs#L334-L340](#)

### Description:

The `validate_and_reconstruct_packet` function supports two mutually exclusive modes for supplying payloads:

1. **inline** - payloads embedded directly in the packet
2. **chunked** - payloads assembled from on-chain chunk accounts via metadata

A guard at the top of the function is meant to enforce this mutual exclusion:

```
337: require!(  
338:     !(has_inline_payloads && has_chunked_metadata),  
339:     RouterError::InvalidPayloadCount  
340: );
```

This check uses NAND logic ("not AND"), which only rejects the case where both modes are active simultaneously. It silently permits the case where `_neither_` mode is active. In that scenario, the function falls into the chunked branch, iterates over empty or zero-chunk metadata, and returns a reconstructed packet with an empty payloads vector.

All three callers (`recv_packet`, `ack_packet`, `timeout_packet`) immediately pass the result to `get_single_payload`, which **rejects** packets that do not have exactly one payload.

This downstream check prevents the empty-payload packet from being processed further. However, `validate_and_reconstruct_packet` itself does not enforce the fundamental invariant that at least one delivery mode must be specified.

A proper XOR check (`has_inline_payloads ^ has_chunked_metadata`) would reject both the conflicting-modes case and the no-modes case, making the validation self-contained rather than dependent on callers to catch the gap.

## Recommendations:

Consider replacing the NAND guard with an XOR check to enforce that exactly one of the inline or chunked mode is active:

```
require!(  
  has_inline_payloads ^ has_chunked_metadata,  
  RouterError::InvalidPayloadCount  
);
```

**Cosmos Labs:** Resolved with [@0e53fbc9a3...](#)

**Zenith:** Verified.

## [I-6] IFT\_PORT\_ID and IFT\_VERSION constants are defined but never used

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [programs/solana/programs/ift/src/constants.rs#L3-L7](#)

### Description:

IFT\_PORT\_ID = "iftport" and IFT\_VERSION = "ift-1" are declared but referenced nowhere.

IFT does not register as a standalone IBC app, it runs over `ics27-gmp`. Channel negotiation, port binding, and version handshaking all belong to GMP, hence the constants can be safely removed without affecting functionality.

### Recommendations:

Consider removing both constants.

**Cosmos Labs:** Resolved with [@d7f4d6c900 ...](#)

**Zenith:** Verified.

## [I-7] IBC app callback failures cannot be converted into universal error acknowledgements

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [recv\\_packet.rs](#)

### Description:

On `recv_packet`, the router calls the IBC app's `on_recv_packet` via CPI and attempts to conceptually fall back to the universal error acknowledgement if the callback fails, but on Solana this is impossible: any CPI failure aborts the entire transaction and the error branch in the router is effectively unreachable.

As a result, if an IBC app is buggy or malicious and consistently errors in `on_recv_packet`, the router can neither write a success ack nor a universal error ack, leaving the packet unacknowledged on the destination chain.

This creates a liveness / economic DoS vector: a relayer, seeing no acknowledgement, may repeatedly re-submit the same `recv_packet` transaction in an attempt to progress the packet and instead incur repeated transaction fees with no way for the router to convert the failure into a final error acknowledgement; in the worst case this can wedge channels until timeouts are processed (if supported) and drain relayer funds via futile retries. This also deviates from the Eureka / Ethereum behavior where router logic can catch app errors and emit the universal error ack, ensuring packets are always terminally processed.

### Recommendations:

We recommend clearly documenting this Solana-specific limitation at the protocol and relayer levels, emphasizing that IBC app callbacks are effectively “all-or-nothing” and that universal error acknowledgements cannot be synthesized by the router when an app reverts.

Operationally, relayers should implement retry/backoff and failure thresholds for `recv_packet` (and other app callbacks) to avoid unbounded fee loss when an app persistently fails, and operators should treat IBC apps as high-trust components subject to the same review and testing standards as the router.

**Cosmos Labs:** Resolved with [@ab36399822 ...](#)

**Zenith:** Verified.

## [I-8] `Packet.timeout_timestamp` uses `i64` on Solana while Solidity uses `uint64`, leading to an unsafe cast in the timeout path

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [programs/solana/packages/solana-ibc-types/src/router.rs#L88](#)
- [programs/solana/programs/ics26-router/src/instructions/timeout\\_packet.rs#L157](#)

### Description:

The Solana `Packet` struct defines `timeout_timestamp` as `i64`, contrary to the Solidity implementation `IICS26RouterMsgs.Packet`, which uses the unsigned integer type `uint64`.

In `timeout_packet()`, the `i64` value gets cast to `u64`:

```
156: require!(  
157:     counterparty_timestamp_secs ≥ packet.timeout_timestamp as u64,  
158:     RouterError::InvalidTimeoutTimestamp  
159: );
```

Rust's `as u64` on a negative `i64` wraps silently (no panic), producing a huge value that would cause the timeout check to fail. In practice, `send_packet` and `recv_packet` both require `timeout_timestamp > clock.unix_timestamp`, so negative values are rejected before reaching this cast. Commitment hashing (`to_be_bytes()`) also produces identical output for `i64` and `u64` when the value is positive.

The `i64` choice likely comes from Solana's `Clock::unix_timestamp` type, but the IBC spec and the Solidity side both treat timestamps as unsigned. Using `u64` and a safe cast from the clock value would be more correct.

### Recommendations:

Consider changing `timeout_timestamp` to `u64` in `Packet` and `MsgSendPacket`, and converting `clock.unix_timestamp` via `u64::try_from()` instead of relying on the implicit `as u64` cast.

**Cosmos Labs:** Resolved with [@13f4220493 ...](#)

**Zenith:** Verified.

## [I-9] Missing account reload after error-ack refund in IFT `finalize_transfer`

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [programs/solana/programs/ift/src/instructions/finalize\\_transfer.rs#L179-L187](#)

### Description:

In `finalize_transfer`, the `Timeout` branch calls `mint_to_account` and then reloads both `mint` and `sender_token_account` in [lines 164-165](#). The error acknowledgement branch performs the same `mint_to_account` call but skips the reload. After a CPI mint, the in-memory `Account` structs hold stale data until `reload` is called.

No current code reads from these accounts after the refund, so the inconsistency is harmless today. It becomes a latent footgun if future changes add post-mint logic that depends on up-to-date balances or supply.

### Recommendations:

Consider adding `ctx.accounts.mint.reload()? and ctx.accounts.sender_token_account.reload()? after the mint_to_account call in the error-ack branch to match the timeout branch.`

**Cosmos Labs:** Resolved with [@14730c5511 ...](#)

**Zenith:** Verified.

## [I-10] IBCApp authority field is stored but never used for access control

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

### Target

- [ics26-router/src/state.rs#L41](#)
- [ics26-router/src/instructions/add\\_ibc\\_app.rs#L64](#)

### Description:

The IBCApp account stores an authority pubkey that is set to the caller's key during `add_ibc_app` at [add\\_ibc\\_app.rs#L64](#). However, this field is never used by any instruction. No instruction checks `ibc_app.authority` to gate updates, removal, or any other per-app operation.

All access control in the router goes through `access_manager::require_role` with the `ID_CUSTOMIZER_ROLE`, which is a global role check unrelated to the stored authority value.

### Recommendations:

Consider either removing the authority field from IBCApp if per-app ownership is not intended, or document the design decision if it is meant for future use.

**Cosmos Labs:** Resolved with [@86cfa21307 ...](#)

**Zenith:** Verified.